编写数据包器 RTL

2026/10/27 10:26:05 · 20 条消息（用户 10 · ChatGPT 10） · 原始会话

你

好的，现在让我们开始写difi_data_packetizer 吧

ChatGPT

可以。我们先只做 `difi_data_packetizer` **的第一版 RTL**，不碰 Parser、Context、Version 和 AXI-Lite。目标也先压得很明确：

输入连续的 32-bit I16/Q16 Sample → 按 `samples_per_packet` 切包 → 输出完整的 DIFI 1.1 Standard Flow Signal Data Packet。

按照我们已经冻结的接口规格，Data Packet 固定由 **7 个 Prologue Word + N 个 IQ Payload Word** 构成；第一个 IQ Sample 被接收时，同时锁存 Stream ID、N 和 Timestamp；Sequence 只有在最后一个 Word 真正完成 AXIS handshake 后才递增。 DIFI_1.1_协议实现规格_v1.0 DIFI_RTL_Interface_Spec_v1.0

我建议今天先把它写到"**能综合 + 结构正确**"，然后马上写 testbench 用 4-Sample 向量打通。

1. 先确定我们真正要实现的状态机

我建议第一版只用 5 个状态：

```
IDLE
  | pkt_start
  ▼
WAIT_FIRST
  | 收到第一个 IQ Sample，同时锁存配置和时间
  ▼
SEND_HEADER
  | 依次发送 Word0 ~ Word6
  ▼
SEND_FIRST_SAMPLE
  | 发送事先缓存的第一个 Sample
  |
  ├── N = 1 ──────────────► IDLE
  |
```

```
  ▼
SEND_PAYLOAD
  │ 后续 IQ 直接 AXIS 流式通过
  │
  └── 第 N 个 Sample发送完成 ──► IDLE
```

这里最关键的是：

第一个 Sample 必须缓存，后面的 Sample 不必缓存。

因为我们是在接受第一个 Sample 的那个时钟沿才知道这个 Packet 的时间戳，但这个 Sample 真正输出时必须排在 Word0～Word6 后面。规格也明确要求首 Sample 需要至少一个 32-bit 缓存。 DIFI_RTL_Interface_Spec_v1.0

而 Word0～Word6 输出期间，我们暂时把：

```verilog
s_axis_iq_tready = 1'b0;
```

让上游暂停。

这版不是追求极致吞吐，而是先追求逻辑绝对清楚。

---

2. 先建立公共协议常量文件

目录可以按照我们之前定的：

```
rtl/
├── include/
│   └── difi_v1_1_defs.vh
└── data/
    └── difi_data_packetizer.v
```

`difi_v1_1_defs.vh`

```verilog
`ifndef DIFI_V1_1_DEFS_VH
`define DIFI_V1_1_DEFS_VH

// ============================================================
// DIFI 1.1 common definitions
```

```
// ============================================================

// Signal Data Packet
`define DIFI_DATA_PKT_TYPE       4'h1

// Header fixed fields
`define DIFI_CLASS_ID_PRESENT    1'b1
`define DIFI_PROFILE_BITS        2'b00
`define DIFI_DATA_TSM            1'b0
`define DIFI_TSI_UTC             2'b01
`define DIFI_TSF_REALTIME_PS     2'b10

// Signal Data fixed words
`define DIFI_OUI_WORD            32'h006A_621E
`define DIFI_DATA_CLASS_WORD     32'h0000_0000

// Number of words before IQ payload
`define DIFI_DATA_PROLOGUE_WORDS 16'd7

// Project limits
`define DIFI_DATA_MAX_SAMPLES    16'd256

`endif
```

这样以后 Context、Version、Parser 全部引用同一份协议常量，不在不同 `.v` 文件里复制魔数。这正是我们之前协议规格里冻结的维护原则。 DIFI_1.1_协议实现规格_v1.0

---

3. 第一版 `difi_data_packetizer.v`

下面这版我建议你**先完整看懂，再自己敲进 Vivado**，不要直接复制完就当结束。

verilog

```
`timescale 1ns / 1ps

`include "difi_v1_1_defs.vh"

module difi_data_packetizer (
    input  wire         aclk,
    input  wire         aresetn,

    // ------------------------------------------------------
    // Packet transaction control
    // ------------------------------------------------------
    input  wire         pkt_start,
    output reg          pkt_busy,
```

```
    output reg          pkt_done,

    // ----------------------------------------------------
    // IQ input AXI4-Stream
    // [31:16] = I16
    // [15:0]  = Q16
    // ----------------------------------------------------
    input  wire [31:0]  s_axis_iq_tdata,
    input  wire         s_axis_iq_tvalid,
    output reg          s_axis_iq_tready,

    // ----------------------------------------------------
    // Active configuration
    // ----------------------------------------------------
    input  wire [31:0]  cfg_data_stream_id,
    input  wire [15:0]  cfg_samples_per_packet,

    // ----------------------------------------------------
    // Time source
    // ----------------------------------------------------
    input  wire [31:0]  time_sec,
    input  wire [63:0]  time_ps,
    input  wire         time_valid,

    // ----------------------------------------------------
    // DIFI packet output AXI4-Stream
    // ----------------------------------------------------
    output reg  [31:0]  m_axis_pkt_tdata,
    output reg          m_axis_pkt_tvalid,
    input  wire         m_axis_pkt_tready,
    output wire [3:0]   m_axis_pkt_tkeep,
    output reg          m_axis_pkt_tlast
);

    // ========================================================
    // State definition
    // ========================================================

    localparam ST_IDLE          = 3'd0;
    localparam ST_WAIT_FIRST    = 3'd1;
    localparam ST_SEND_HEADER   = 3'd2;
    localparam ST_SEND_FIRST    = 3'd3;
    localparam ST_SEND_PAYLOAD  = 3'd4;

    reg [2:0] state;

    // ========================================================
    // Internal registers
    // ========================================================

    // 4-bit Signal Data sequence number
```

```
    reg [3:0] data_seq;

    // First sample must be buffered while header is transmitted
    reg [31:0] first_sample_reg;

    // Packet-level snapshots
    reg [31:0] snapshot_stream_id;
    reg [15:0] snapshot_samples;
    reg [31:0] snapshot_time_sec;
    reg [63:0] snapshot_time_ps;

    // Header Word0 ~ Word6 index
    reg [2:0] header_index;

    // Number/index tracking for payload already after first sample
    reg [15:0] payload_count;

    // ========================================================
    // Derived values
    // ========================================================

    wire [15:0] packet_size_words;

    assign packet_size_words =
        snapshot_samples + `DIFI_DATA_PROLOGUE_WORDS;

    // Every output DIFI word is a complete 32-bit word
    assign m_axis_pkt_tkeep = 4'b1111;

    // Optional safety gate:
    // Do not establish a packet while samples_per_packet is illegal.
    wire cfg_samples_valid;

    assign cfg_samples_valid =
        (cfg_samples_per_packet >= 16'd1) &&
        (cfg_samples_per_packet <= `DIFI_DATA_MAX_SAMPLES);

    // ========================================================
    // Output combinational logic
    // ========================================================

    always @(*) begin

        // Defaults
        s_axis_iq_tready = 1'b0;

        m_axis_pkt_tdata  = 32'd0;
        m_axis_pkt_tvalid = 1'b0;
        m_axis_pkt_tlast  = 1'b0;

        case (state)
```

```
// ------------------------------------------------
// Waiting for pkt_start
// ------------------------------------------------
ST_IDLE: begin
    // Nothing transferred here.
end

// ------------------------------------------------
// Wait for first IQ sample.
//
// First sample handshake is also the snapshot point
// for configuration and timestamp.
// ------------------------------------------------
ST_WAIT_FIRST: begin

    if (time_valid && cfg_samples_valid)
        s_axis_iq_tready = 1'b1;
    else
        s_axis_iq_tready = 1'b0;

end

// ------------------------------------------------
// Send DIFI Word0 ~ Word6
// ------------------------------------------------
ST_SEND_HEADER: begin

    m_axis_pkt_tvalid = 1'b1;
    m_axis_pkt_tlast  = 1'b0;

    case (header_index)

        // Word0 : Header
        3'd0: begin
            m_axis_pkt_tdata = {
                `DIFI_DATA_PKT_TYPE,       // [31:28]
                `DIFI_CLASS_ID_PRESENT,    // [27]
                `DIFI_PROFILE_BITS,        // [26:25]
                `DIFI_DATA_TSM,            // [24]
                `DIFI_TSI_UTC,             // [23:22]
                `DIFI_TSF_REALTIME_PS,     // [21:20]
                data_seq,                  // [19:16]
                packet_size_words          // [15:0]
            };
        end

        // Word1 : Stream ID
        3'd1: begin
            m_axis_pkt_tdata =
                snapshot_stream_id;
```

```
                end

                // Word2 : OUI
                3'd2: begin
                    m_axis_pkt_tdata =
                        `DIFI_OUI_WORD;
                end

                // Word3 : ICC / PCC
                3'd3: begin
                    m_axis_pkt_tdata =
                        `DIFI_DATA_CLASS_WORD;
                end

                // Word4 : Integer Timestamp
                3'd4: begin
                    m_axis_pkt_tdata =
                        snapshot_time_sec;
                end

                // Word5 : Fractional Timestamp [63:32]
                3'd5: begin
                    m_axis_pkt_tdata =
                        snapshot_time_ps[63:32];
                end

                // Word6 : Fractional Timestamp [31:0]
                3'd6: begin
                    m_axis_pkt_tdata =
                        snapshot_time_ps[31:0];
                end

                default: begin
                    m_axis_pkt_tdata = 32'd0;
                end

            endcase
        end

        // ------------------------------------------------
        // Send the buffered first IQ sample
        // ------------------------------------------------
        ST_SEND_FIRST: begin

            m_axis_pkt_tdata  = first_sample_reg;
            m_axis_pkt_tvalid = 1'b1;

            if (snapshot_samples == 16'd1)
                m_axis_pkt_tlast = 1'b1;
            else
                m_axis_pkt_tlast = 1'b0;
```

```
        end

        // ------------------------------------------------
        // Stream remaining IQ samples directly through
        // ------------------------------------------------
        ST_SEND_PAYLOAD: begin

            m_axis_pkt_tdata  = s_axis_iq_tdata;
            m_axis_pkt_tvalid = s_axis_iq_tvalid;

            // Backpressure propagates directly upstream
            s_axis_iq_tready = m_axis_pkt_tready;

            if (payload_count == snapshot_samples - 16'd1)
                m_axis_pkt_tlast = 1'b1;
            else
                m_axis_pkt_tlast = 1'b0;

        end

        default: begin
            s_axis_iq_tready  = 1'b0;
            m_axis_pkt_tvalid = 1'b0;
            m_axis_pkt_tlast  = 1'b0;
            m_axis_pkt_tdata  = 32'd0;
        end

    endcase
end

// ============================================================
// Sequential logic
// ============================================================

always @(posedge aclk) begin

    if (!aresetn) begin

        state               <= ST_IDLE;

        pkt_busy            <= 1'b0;
        pkt_done            <= 1'b0;

        data_seq            <= 4'd0;

        first_sample_reg    <= 32'd0;

        snapshot_stream_id  <= 32'd0;
        snapshot_samples    <= 16'd0;
        snapshot_time_sec   <= 32'd0;
```

```
            snapshot_time_ps     <= 64'd0;

            header_index         <= 3'd0;
            payload_count        <= 16'd0;

        end
        else begin

            // pkt_done is an event pulse
            pkt_done <= 1'b0;

            case (state)

                // ------------------------------------------------
                // IDLE
                // ------------------------------------------------
                ST_IDLE: begin

                    pkt_busy <= 1'b0;

                    if (pkt_start) begin
                        pkt_busy <= 1'b1;
                        state    <= ST_WAIT_FIRST;
                    end

                end

                // ------------------------------------------------
                // Wait for first sample
                // ------------------------------------------------
                ST_WAIT_FIRST: begin

                    if (s_axis_iq_tvalid &&
                        s_axis_iq_tready) begin

                        // First IQ sample
                        first_sample_reg <=
                            s_axis_iq_tdata;

                        // Snapshot configuration
                        snapshot_stream_id <=
                            cfg_data_stream_id;

                        snapshot_samples <=
                            cfg_samples_per_packet;

                        // Snapshot timestamp
                        snapshot_time_sec <=
                            time_sec;

                        snapshot_time_ps <=
```

```
                time_ps;

            // Start from Word0
            header_index <= 3'd0;

            // No payload word has been sent yet
            payload_count <= 16'd0;

            state <= ST_SEND_HEADER;

        end
    end

    // ----------------------------------------------
    // Header Word0 ~ Word6
    // ----------------------------------------------
    ST_SEND_HEADER: begin

        if (m_axis_pkt_tvalid &&
            m_axis_pkt_tready) begin

            if (header_index == 3'd6) begin

                state <= ST_SEND_FIRST;

            end
            else begin

                header_index <=
                    header_index + 3'd1;

            end
        end
    end

    // ----------------------------------------------
    // Buffered first sample
    // ----------------------------------------------
    ST_SEND_FIRST: begin

        if (m_axis_pkt_tvalid &&
            m_axis_pkt_tready) begin

            if (snapshot_samples == 16'd1) begin

                // Entire packet completed
                data_seq <= data_seq + 4'd1;

                pkt_busy <= 1'b0;
                pkt_done <= 1'b1;
```

```
                state <= ST_IDLE;

            end
            else begin

                // Sample 0 has been transmitted.
                // Next incoming sample is sample 1.
                payload_count <= 16'd1;

                state <= ST_SEND_PAYLOAD;

            end
        end
    end

    // ------------------------------------------------
    // Remaining samples
    // ------------------------------------------------
    ST_SEND_PAYLOAD: begin

        if (m_axis_pkt_tvalid &&
            m_axis_pkt_tready) begin

            if (payload_count ==
                snapshot_samples - 16'd1) begin

                // Last payload word completed
                data_seq <= data_seq + 4'd1;

                pkt_busy <= 1'b0;
                pkt_done <= 1'b1;

                state <= ST_IDLE;

            end
            else begin

                payload_count <=
                    payload_count + 16'd1;

            end
        end
    end

    default: begin

        state    <= ST_IDLE;
        pkt_busy <= 1'b0;

    end
```

```
            endcase
        end
    end

endmodule
```

---

4. 现在先理解最核心的地方

这段代码真正需要你吃透的不是 Header 拼接，而是下面四个地方。

第一，第一个 Sample 为什么不能直接透传?

因为：

```
IQ0 handshake
    ↓
此时才能锁存：
Stream ID
Samples per packet
time_sec
time_ps
    ↓
之后才能生成：
Word0
Word1
...
Word6
    ↓
最后才轮到 IQ0
```

所以必须：

```verilog
first_sample_reg <= s_axis_iq_tdata;
```

等 Header 发完以后再：

```verilog
m_axis_pkt_tdata = first_sample_reg;
```

这就是 `ST_SEND_FIRST` 存在的原因。

---

5. Header 的构造你应该能自己推出来

这里：

verilog

```verilog
m_axis_pkt_tdata = {
    4'h1,
    1'b1,
    2'b00,
    1'b0,
    2'b01,
    2'b10,
    data_seq,
    packet_size_words
};
```

对应：

纯文本

```
31                  28
┌─────────────────────┐
│ Packet Type = 0x1   │
├─────────────────────┤
│ C = 1               │ bit27
├─────────────────────┤
│ Reserved = 00       │ bit26:25
├─────────────────────┤
│ TSM = 0             │ bit24
├─────────────────────┤
│ TSI = 01            │ bit23:22
├─────────────────────┤
│ TSF = 10            │ bit21:20
├─────────────────────┤
│ Seq                 │ bit19:16
├─────────────────────┤
│ Packet Size         │ bit15:0
└─────────────────────┘
```

这和已经冻结的 DIFI Data Header 完全一致。 DIFI_1.1_协议实现规格_v1.0

例如：

```
Seq = 0
N = 4
Packet Size = 7 + 4 = 11 = 0x000B
```

得到：

```
Word0 = 0x1860000B
```

正好就是我们规格附录 C 的测试向量。

---

6. `SEND_PAYLOAD` 这几行特别重要

```verilog
m_axis_pkt_tdata  = s_axis_iq_tdata;
m_axis_pkt_tvalid = s_axis_iq_tvalid;

s_axis_iq_tready = m_axis_pkt_tready;
```

实际上这里是在做：

```
           difi_data_packetizer
        ┌──────────────────────┐
        │                      │
IQ ─────┼────── direct path ────┼──► DIFI Payload
        │                      │
        └──────────────────────┘
```

也就是说从第二个 Sample 开始，不再需要存起来。

AXIS backpressure 也天然传播：

```
m_axis_pkt_tready = 0
        ↓
s_axis_iq_tready = 0
```

```
        ↓
上游必须停住当前 Sample
```

所以只要上游遵守 AXI4-Stream：

`TVALID=1 && TREADY=0` 时保持 `TDATA` 不变，

我们的输出也会保持不变。

这正符合接口规格里最重要的 AXIS 不变量：**只有 `TVALID && TREADY` 才允许状态推进。**

DIFI_RTL_Interface_Spec_v1.0

---

7. `payload_count` 到底在数什么？

我这里专门规定：

```
payload_count = 当前准备传输的 Sample 序号
```

Sample 从 0 开始。

例如 N=4：

```
Sample 0
    first_sample_reg
    ↓
SEND_FIRST
```

发送 Sample0 后：

```verilog
payload_count <= 1;
```

之后：

```
payload_count = 1 → Sample1
payload_count = 2 → Sample2
payload_count = 3 → Sample3
```

最后条件：

```verilog
payload_count == snapshot_samples - 1
```

即：

```
3 == 4 - 1
```

所以 Sample3 上：

```verilog
m_axis_pkt_tlast = 1;
```

完全正确。

---

8. 为什么 `data_seq` 只能在这里加？

我们绝不能在：

```
pkt_start
```

时加。

也不能在：

```
生成 TLAST
```

时加。

而只能在：

```verilog
```

```
m_axis_pkt_tvalid &&
m_axis_pkt_tready &&
m_axis_pkt_tlast
```

真正发生时增加。

所以代码中最后一个 Sample handshake 后：

```verilog
data_seq <= data_seq + 4'd1;
```

如果此时：

```
TVALID = 1
TLAST  = 1
TREADY = 0
```

就什么都不做：

```
TDATA 保持
TLAST 保持
Seq 不增加
FSM 不前进
```

这也是我们之前冻结的明确要求。 DIFI_RTL_Interface_Spec_v1.0

---

9. 这版有一个刻意做出的性能取舍

当前：

```
接收 Sample0
    ↓
停止 IQ 输入
    ↓
发送 7 个 Header Word
    ↓
发送 Sample0
```

```
    ↓
恢复接收 Sample1...
```

因此理想情况下，每个 Packet 都会让 IQ 输入暂停至少：

```
7 + 1 ≈ 8 clocks
```

这个设计**功能上没问题，但吞吐率不是最终最优版本。**

以后可以改成：

```
IQ → 小 FIFO → Packetizer
```

这样 Header 输出期间，IQ 仍然可以继续流入 FIFO。

甚至进一步：

```
IQ FIFO
   ↓
Packetizer
   ↓
Data Packet
```

但这是之后性能优化的问题。接口规格本身就允许第一版在 Word0 ~ 6 期间拉低 IQ `TREADY`，因此我们目前没有违背架构约定。 DIFI_RTL_Interface_Spec_v1.0

---

10. 现在先别急着写 Parser

下一步应该直接写：

```
tb_data_packetizer.v
```

第一项测试就用我们已经冻结的这个向量：

```
```

```
Stream ID = 0x00000001
Seq       = 0
Time      = 100 s + 0 ps
N         = 4

Samples:

I=1       Q=-1
I=32767   Q=-32768
I=-2      Q=3
I=0       Q=0
```

期望完整输出：

纯文本

```
Word0   1860000B
Word1   00000001
Word2   006A621E
Word3   00000000
Word4   00000064
Word5   00000000
Word6   00000000
Word7   0001FFFF
Word8   7FFF8000
Word9   FFFE0003
Word10  00000000   ← TLAST=1
```

这正是协议规格中给出的 Signal Data 参考测试向量。 DIFI_1.1_协议实现规格_v1.0

所以我们现在最合理的下一步，就是不要继续往 RTL 里堆功能，而是马上给这个模块写第一个 testbench。 一旦这个 11 Word 的包能在 Vivado Simulation 里逐 Word 正确输出，我们的 `difi_data_packetizer` 第一块核心逻辑实际上就已经真正跑起来了。

你

你不用直接把完整的代码写出来，你可以参考我们之前写的规范文档，告诉我这一版 difi_data_packetizer 要实现什么功能，需要注意什么等，主要还是我来写

ChatGPT

可以，这样更合适。`difi_data_packetizer` 本来就很适合作为你真正开始写协议 RTL 的第一个模块：协议结构不复杂，但会把 **AXI4-Stream 握手、状态机、配置快照、计数器、backpressure、Packet 边界** 这些后面反复用到的东西都练一遍。

这一版我们严格按照已经冻结的《DIFI 1.1 协议实现规格 v1.0》和《DIFI 1.1 RTL 模块边界与接口规格 v1.0》来，不临时往里面加功能。 DIFI_1.1_协议实现规格_v1.0 DIFI_RTL_Interface_Spec_v1.0

一、这一版 `difi_data_packetizer` 到底要干什么

它的职责可以压缩成一句话：

> **每收到一次发包请求，就从连续 IQ Stream 中取指定数量的 I16/Q16 Sample，在前面加上 DIFI Signal Data Packet 的 7 个固定/半固定 Word，然后作为一个完整 AXI4-Stream Packet 输出。**

输入是：

```
连续 IQ：
Sample0
Sample1
Sample2
Sample3
...
```

其中每个 Sample 已经整理成：

```
31            16 15             0
+----------------+----------------+
|      I16       |      Q16       |
+----------------+----------------+
```

输出则变成：

```
Word0   DIFI Header
Word1   Stream ID
Word2   OUI
Word3   ICC / PCC
Word4   Integer Timestamp
Word5   Fraction Timestamp[63:32]
Word6   Fraction Timestamp[31:0]
Word7   Sample0
Word8   Sample1
...
        Sample N-1 + TLAST
```

因此：

```
Packet Size = 7 + N
```

这里一个 IQ Sample 恰好一个 32-bit Word，这是我们固定 I16/Q16 格式以后最大的简化之一。

DIFI_1.1_协议实现规格_v1.0

二、你真正要实现的功能可以拆成这几块

功能块	这一版必须做什么
发包控制	`pkt_start` 发起一次 Data Packet
IQ 接收	通过 `s_axis_iq_*` 接收 N 个 I/Q Sample
配置快照	每个 Packet 锁存 Stream ID 和 samples_per_packet
时间戳快照	第一个 Sample 被接受时锁存 `time_sec/time_ps`
Header 生成	正确产生 Word0 ~ Word6
Payload 输出	按原顺序发送 N 个 IQ Sample
Packet Size	自动生成 `7 + N`
Sequence	每完成一个完整 Packet 后 `+1 mod 16`
AXIS 输出	正确处理 `TVALID/TREADY/TLAST/TKEEP`
Backpressure	下游停顿时不能丢 Word、跳 Word 或提前改变状态
事务状态	正确产生 `pkt_busy`、`pkt_done`

除此之外都不要做。

尤其**不要**在这里做：

```
Context Packet
Version Packet
```

```
Context/Data 调度
AXI4-Lite 地址译码
UDP Header
IP Header
Ethernet Header
时间同步
错误计数寄存器
```

这些已经明确划给其他模块。 DIFI_RTL_Interface_Spec_v1.0

三、最关键的问题：一个 Packet 是什么时候真正“开始”的？

这是你写 RTL 前最需要想明白的地方。

我们已经规定：

```
pkt_start
    ↓
Packetizer 知道“我要准备发一个包了”
    ↓
等待第一个 IQ Sample
    ↓
第一个 Sample 真正 handshake
    ↓
同时锁存：
    first Sample
    Stream ID
    samples_per_packet
    time_sec
    time_ps
    ↓
这个 Packet 正式建立
```

也就是说：

`pkt_start` 时刻 ≠ Packet 时间戳时刻。

真正决定 Data Packet 时间戳的是：

```
第一个 IQ Sample 被接受的那个时钟沿
```

规范里已经明确冻结了这一点。 DIFI_RTL_Interface_Spec_v1.0

所以你写的时候最好把两件事情分开理解：

```
pkt_start：
“我要开始准备一个 Data Packet”

first IQ handshake：
“这个 Data Packet 的具体内容正式确定”
```

四、为什么第一个 IQ Sample 必须特殊处理

这个模块里最有意思的地方其实不是 Header，而是 **Sample0**。

假设：

```
Sample0 = 第一个 IQ Sample
```

你只有真正接受 Sample0 时，才能同时锁存：

```
timestamp
Stream ID
N
```

但是输出顺序又要求：

```
Word0
Word1
...
Word6
Sample0
```

因此 Sample0 不能收到以后直接发出去。

必须先：

```
接受 Sample0
      ↓
```

```
保存 Sample0
      ↓
发送 Word0~Word6
      ↓
再输出保存下来的 Sample0
```

所以至少需要：

```
first_sample_reg
```

这样一个 32-bit 寄存器。

这是第一版 Packetizer 里非常重要的一个设计点。 DIFI_RTL_Interface_Spec_v1.0

五、这一版不要求你把整个 IQ 流缓存起来

按照我们的第一版设计：

```
Sample0 → 缓存
```

但是：

```
Sample1
Sample2
Sample3
...
```

可以在进入 Payload 阶段以后直接流式转发。

大概是：

```
              +----------------------+
Sample0 ----->| first_sample_reg     |
              +----------------------+
                        |
                        v
                  Payload Word0
```

```
Sample1 --------------------------> Payload Word1
Sample2 --------------------------> Payload Word2
Sample3 --------------------------> Payload Word3
```

这样第一版内部只需要一个 Sample 缓冲，而不用现在就做 256×32 FIFO。

以后为了提高吞吐率，可以在 Packetizer 前面加 FIFO，这不是当前第一版必须解决的问题。

六、你可以按什么状态来设计 FSM

我不直接给你状态机代码，但我建议你先画成类似这样的逻辑：

‹/› 纯文本

```
                pkt_start
      IDLE --------------------> WAIT_FIRST
                                    |
                                    | first IQ handshake
                                    |
                                    v
                               SEND_HEADER
                                    |
                                    | Word0~Word6 完成
                                    |
                                    v
                               SEND_FIRST
                                  /     \
                             N=1/        \N>1
                               /          \
                            DONE        SEND_PAYLOAD
                                          |
                                          | N个Sample完成
                                          |
                                          v
                                         DONE
```

你甚至不一定需要单独的 `DONE` 状态，因为最后一个 Payload handshake 当拍可以：

‹/› 纯文本

```
pkt_done <= 1
pkt_busy <= 0
seq      <= seq + 1
state    <= IDLE
```

具体状态怎么编码是你自己的实现自由。

规范只冻结外部行为，没有要求你一定用几个状态。 DIFI_RTL_Interface_Spec_v1.0

七、Word0 ~ Word6 里哪些是动态的，哪些是固定的

你写 Header Generator 时，可以直接按照这个表检查：

Word	内容	来源
Word0	Header	固定字段 + `data_seq` + `7+N`
Word1	Stream ID	本 Packet 锁存的 `cfg_data_stream_id`
Word2	OUI	`32'h006A621E`
Word3	ICC/PCC	`32'h00000000`
Word4	Integer Timestamp	锁存的 `time_sec`
Word5	Fraction Timestamp High	锁存的 `time_ps[63:32]`
Word6	Fraction Timestamp Low	锁存的 `time_ps[31:0]`

Word0 的字段则是：

```
Packet Type = 1
C           = 1
bits26:25   = 00
TSM         = 0
TSI         = 01
TSF         = 10
Seq         = data_seq
Size        = 7 + N
```

例如：

```
Seq = 0
N   = 4
```

则：

```
Size = 11 = 0x000B
Word0 = 0x1860000B
```

这个值你之后仿真第一眼就应该检查。 DIFI_1.1_协议实现规格_v1.0

八、这里一定不要混淆“大端”和 RTL TDATA

Packetizer 输出：

```
m_axis_pkt_tdata = 32'h1234_ABCD
```

就代表协议 Word 数值：

```
32'h1234_ABCD
```

你现在不要在 Packetizer 内做：

```
1234ABCD → CDAB3412
```

之类的 byte swap。

我们已经冻结：

```
RTL 的 32-bit TDATA = 协议 Word 数值
```

未来进入 UDP byte stream 时才对应：

```
12 34 AB CD
```

字节序列化属于后面的网络边界，不属于 `difi_data_packetizer`。 DIFI_RTL_Interface_Spec_v1.0

这一点以后非常容易踩坑。

九、AXI4-Stream 是这一版最重要的实现原则

你现在写的时候可以一直用这一条检查自己的代码：

> **只有 `TVALID && TREADY` 同时为 1 的那个时钟沿，才能认为一个 Word/Sample 真正被传输了。**

例如你正在发送：

```
纯文本
Word3 = ICC/PCC
```

此时：

```
纯文本
m_axis_pkt_tvalid = 1
m_axis_pkt_tready = 0
```

那么下一拍必须还是：

```
纯文本
Word3
```

不能变成 Word4。

因此不能：

```
纯文本
只要 TVALID=1
word_cnt++
```

必须是：

```
纯文本
if (m_axis_pkt_tvalid && m_axis_pkt_tready)
    word_cnt++;
```

同理 Payload 计数器、Header Index、Sequence 都必须遵守这个原则。
DIFI_RTL_Interface_Spec_v1.0

十、Payload 阶段 backpressure 如何传播

进入正常 Payload 阶段以后，逻辑关系实际上比较简单。

如果：

```
IQ Source
   ↓
Packetizer
   ↓
Packet Output
```

那么大致应该表现成：

```
下游 Packet 接不动
m_axis_pkt_tready = 0
        ↓
Packetizer 当前 IQ 不能被真正送出
        ↓
s_axis_iq_tready = 0
        ↓
IQ Source 停住
```

这样就不会偷偷吃掉一个 IQ Sample。

因为我们的接口契约明确假定：

> Packetizer 可以向 IQ Source 施加 backpressure。

如果未来接的是 ADC/DDC 那种不可暂停的持续实时源，就必须在 Packetizer 前增加 FIFO，而不是让 Packetizer 静默丢 Sample。 DIFI_RTL_Interface_Spec_v1.0

十一、`TLAST` 的产生不是来自 IQ 输入

注意 IQ 输入接口没有：

```
纯文本
s_axis_iq_tlast
```

这是故意的。

因为 IQ 本身是连续数据流：

```
纯文本
... IQ IQ IQ IQ IQ IQ IQ ...
```

它不知道什么叫一个 DIFI Packet。

Packetizer 自己根据：

```
纯文本
snapshot_samples_per_packet
```

切割：

```
纯文本
每 N 个 Sample → 一个 Packet
```

所以：

```
纯文本
N = 4
```

那么：

```
纯文本
Sample0 TLAST=0
Sample1 TLAST=0
Sample2 TLAST=0
Sample3 TLAST=1
```

这里的 Packet 边界完全属于 Packetizer。 DIFI_RTL_Interface_Spec_v1.0

十二、Sequence Number 的更新时机尤其要小心

`data_seq`：

```
纯文本
0,1,2,...,14,15,0,1...
```

4 bit 自动回卷即可。

但是它只能在：

```
纯文本
整个 Packet 的最后一个 Word
```

真正 handshake 后更新。

也就是：

```
纯文本
m_axis_pkt_tvalid = 1
m_axis_pkt_tready = 1
m_axis_pkt_tlast  = 1
```

这一拍之后：

```
纯文本
data_seq = data_seq + 1
```

如果：

```
纯文本
TVALID = 1
TLAST  = 1
TREADY = 0
```

那么 Packet **还没有发送完成。**

此时：

```
纯文本
```

```
Sequence 不变
TLAST 不变
TDATA 不变
状态不变
```

这是之后做 backpressure 仿真时的重点测试项。

十三、`pkt_busy` 和 `pkt_done` 应该怎么理解

按照我们的接口定义：

```
pkt_busy
```

是一个**状态**。

从接受：

```
pkt_start
```

以后开始为 1，一直到最后一个 Packet Word 成功 handshake。

而：

```
pkt_done
```

是一个**事件脉冲**：

```
0 0 0 0 1 0 0
      ^
最后一 Word handshake
```

只保持一拍。

以后 Scheduler 就可以看：

```
```

```
pkt_busy
```

知道 Data Packetizer 是否被占用；

看：

```
纯文本
pkt_done
```

知道刚刚一个 Data Packet 发完了。

十四、配置一定要 Snapshot，不要直接一直读 cfg

假设一个包：

```
纯文本
N = 256
Stream ID = 1
```

正在发送过程中，上位机突然把配置改成：

```
纯文本
N = 128
Stream ID = 7
```

当前 Packet 绝不能发送到一半突然变成 Stream ID=7，也不能 Payload 发到 128 个就停止。

因此当前 Packet 使用：

```
纯文本
snapshot_stream_id
snapshot_samples
snapshot_time_sec
snapshot_time_ps
```

而不是始终直接使用：

```
纯文本
cfg_data_stream_id
cfg_samples_per_packet
```

```
time_sec
time_ps
```

Snapshot 以后：

```
外部输入怎么改
      ↓
当前 Packet 不受影响
      ↓
下一 Packet 再使用新值
```

这是协议包一致性的基础。 DIFI_1.1_协议实现规格_v1.0

十五、`time_valid` 怎么处理

我们当前冻结的规则是：

> `time_valid=0` 时不能接受一个 Packet 的第一个 Sample。

因为只要第一个 Sample 接受下来，就意味着这个 Data Packet 的 Timestamp 被确定了。

所以在：

```
WAIT_FIRST
```

阶段，你需要同时满足：

```
存在有效 IQ Sample
时间源有效
Packetizer 准备好接收
```

才建立 Packet。

但是 Packet 一旦建立，之后：

```
time_valid
time_sec
```

```
time_ps
```

再怎么变化都不影响当前包，因为已经 snapshot 了。 DIFI_RTL_Interface_Spec_v1.0

十六、`samples_per_packet` 有一个需要你注意的边界问题

协议实现规格已经规定：

```
纯文本
1 <= samples_per_packet <= 256
```

默认建议：

```
纯文本
256
```

这样：

```
纯文本
Packet Size = 263 words
            = 1052 bytes
```

普通 MTU=1500 下不会产生 IP fragmentation。 DIFI_1.1_协议实现规格_v1.0

但是目前的接口文档**没有给 Packetizer 定义专门的非法配置 error 输出。**

也就是说，如果：

```
纯文本
cfg_samples_per_packet = 0
```

或者：

```
纯文本
300
```

文档已经说明它是非法配置，但还没有冻结“Packetizer 应该具体如何报告”。

所以你现在写 RTL 时不要擅自搞一个：

```
error_samples_invalid
```

新端口出来。

第一版可以先把：

```
1~256
```

作为模块使用前提，并在 testbench 只输入合法值。等后面做配置寄存器模块时，再统一决定非法配置的拒绝/钳位/状态报告方式。

这是目前规范里少数**合法范围明确，但非法行为尚未接口化**的地方。

十七、你写完以后可以拿这几个不变量自查

你不需要一开始想着“代码有没有写漂亮”，先保证下面这些永远成立：

```
AXIS 不 handshake → FSM/计数器不能推进。

当前 Packet 的 Stream ID / N / Timestamp
→ Packet 中途永远不变。

IQ Sample 顺序：
输入 Sample0,1,2,3
→ 输出 Payload 必须仍然是 0,1,2,3。

Packet Size
→ 必须严格等于实际输出 Word 数。

最后一个 Payload
→ 必须且只能 TLAST=1。

data_seq
→ 只能在最后一 Word handshake 后 +1。

Word0~6 期间
→ 不能误吃 Sample1、Sample2...

pkt_busy=1 时
→ 新 pkt_start 不得启动第二个 Packet。
```

如果这些都满足，第一版的主体基本就稳了。

你现在实际可以从哪里开始写

你现在先不要一上来写完整 `always` 块。

建议你第一步只在纸上或者代码注释里把下面三个东西确定下来：

```
纯文本
1. FSM 有哪些状态，各状态什么时候跳转；

2. 当前 Packet 需要哪些 snapshot 寄存器；

3. 需要哪些计数器：
   header_index？
   payload_index？
   是否能合并成一个 word_index？
```

其中第 3 点我反而建议你自己思考一下。

比如可以有两种思路：

```
纯文本
方案 A：
header_index + payload_count
```

也可以：

```
纯文本
方案 B：
统一 word_count
Word0~Word6
Word7~...
```

两种都可以实现，接口规范没有限制内部实现。

你可以先自己写出状态机规划和准备使用的内部寄存器，不需要写 RTL。发给我以后，我先帮你做一次设计审查，看看状态划分和计数逻辑有没有隐患，再开始真正写 Verilog。

你

虽然我知道整体这个模块要干什么，但涉及到直接去写，我感觉有些困难，可能是因为我好久没写这种多模块的RTL了，三段式还是五段式状态机写法什么的都不熟悉了，我该怎么办？请你一步一步教我彻底理解这个模块，助我把它写出来

ChatGPT

可以。你现在卡住的点并不是"不理解 DIFI"，而是**从"知道功能"到"把功能拆成时序 RTL"这一层有点生疏了**。这很正常，尤其你前面一直在做协议学习，突然切到 FSM、握手、寄存器、时序，脑子里需要重新建立 RTL 的组织方式。

我们不要一上来写完整代码。接下来我会把 `difi_data_packetizer` 当成一个教学例子，带你从最基础的 **"一个时钟沿到底发生什么"** 开始，最后让你自己把完整模块写出来。

我们已经冻结的模块行为是：输入 I16/Q16 AXIS，收到 `pkt_start` 后等待第一个 Sample；第一个 Sample handshake 时锁存 Sample、Stream ID、N 和时间戳；然后输出 Word0~6，再输出 N 个 IQ Sample；最后一个 Payload Word handshake 后 `seq+1`、`pkt_done`。
DIFI_RTL_Interface_Spec_v1.0 协议上的 Data Packet 则固定是 `7 + N` 个 32-bit Word。
DIFI_1.1_协议实现规格_v1.0

先解决你对"状态机写法"的困惑

你可能把几个概念混在一起了。

RTL 状态机最常见的是：

```
纯文本
一段式 FSM
二段式 FSM
三段式 FSM
```

你说的"五段式"，更常见的是 CPU 的"五级流水线"，不是这里必须掌握的状态机写法。

对我们这个项目，我建议你重新从 **二段式状态机** 开始。

为什么？

因为它足够清楚，又不会把代码拆得太碎。

基本结构就是：

```verilog
// ① 时序逻辑
always @(posedge aclk) begin
    // 状态、计数器、寄存器更新
end

// ② 组合逻辑
always @(*) begin
    // 根据当前 state / counter
    // 决定 TDATA、TVALID、TREADY、TLAST
end
```

你暂时不用纠结“一段式和三段式哪个更规范”。

我们这个模块就按：

一个时序 always + 一个组合 always

来写。

第一课：RTL 里到底有什么东西？

你现在先暂时忘掉 DIFI。

一个同步 RTL 模块，核心其实只有两类东西。

第一类：有记忆的东西

例如：

```纯文本
当前状态
Sequence Number
已经发到第几个 Header Word
已经发了多少个 Sample
第一个 Sample
锁存后的 Stream ID
锁存后的 Timestamp
```

这些东西**跨时钟周期存在**。

所以它们必须是寄存器：

```verilog
reg [2:0] state;
reg [3:0] data_seq;
reg [2:0] header_index;
reg [15:0] payload_count;

reg [31:0] first_sample_reg;
reg [31:0] snapshot_stream_id;
reg [15:0] snapshot_samples;
reg [31:0] snapshot_time_sec;
reg [63:0] snapshot_time_ps;
```

你可以记一句：

凡是“下一拍还要记得”的信息，都应该想到寄存器。

第二类：当前状态直接决定的东西

例如现在状态是：

```纯文本
SEND_HEADER
```

并且：

```纯文本
header_index = 2
```

那么当前输出就应该是：

```纯文本
Word2 = OUI
```

这种东西不需要额外“记住”。

可以直接：

```verilog
always @(*) begin
    case (state)
```

```
        ...
    endcase
end
```

根据状态算出来。

所以你会逐渐发现：

```
寄存器 = 记忆
组合逻辑 = 根据当前记忆作决定
```

这就是 FSM RTL 的本质。

第二课：我们先完全不用 Verilog 描述 Packetizer

假设：

```
N = 4

IQ 输入：
A
B
C
D
```

我们希望 Packetizer 做：

```
pkt_start

等待 A

收到 A
↓
记住 A
记住时间
记住 N=4
记住 Stream ID

输出：
```

```
Word0
Word1
Word2
Word3
Word4
Word5
Word6
A
B
C
D
```

这就是整个模块。

你现在甚至可以想象自己是一个人坐在桌子前工作：

“有人告诉我开始打包。”

你说：

“好，我先等第一个 IQ。”

第一个 IQ 来了。

你说：

“先别往外发，我记下来。”

然后你开始写：

```
纯文本
Header
Stream ID
OUI
Class
Timestamp...
```

写完以后：

“好了，现在把刚才存起来的 IQ0 发出去。”

然后再接：

```
纯文本
IQ1
IQ2
```

```
IQ3
```

直到 N 个 Sample 发完。

这就是状态机。

第三课：为什么一定需要“状态”？

因为模块在不同时间要干不同的事。

比如当前周期可能正在：

```纯文本
等待 pkt_start
```

下一段时间可能：

```纯文本
等待第一个 Sample
```

再下一段时间：

```纯文本
发 Header
```

再下一段时间：

```纯文本
发 Payload
```

所以必须有一个变量告诉硬件：

“我现在正在干什么？”

这个变量就是：

```verilog
state
```

例如：

```verilog
localparam ST_IDLE         = 3'd0;
localparam ST_WAIT_FIRST   = 3'd1;
localparam ST_SEND_HEADER  = 3'd2;
localparam ST_SEND_FIRST   = 3'd3;
localparam ST_SEND_PAYLOAD = 3'd4;
```

这里 `state` 并不是什么神秘东西。

例如：

```纯文本
state = 3
```

只是在说：

> 我目前处于 SEND_FIRST。

第四课：我们逐个状态想，不写代码

状态 1：IDLE

这个状态就是：

> 我没在发包。

所以：

```纯文本
pkt_busy = 0
```

这个时候唯一关注的输入：

```纯文本
pkt_start
```

如果：

```
pkt_start = 0
```

继续：

```
IDLE
```

如果：

```
pkt_start = 1
```

下一拍进入：

```
WAIT_FIRST
```

同时：

```
pkt_busy = 1
```

所以状态转移：

```
IDLE
  |
  | pkt_start
  v
WAIT_FIRST
```

非常简单。

第五课：WAIT_FIRST 才是真正的 Packet 起点

进入：

```
纯文本
WAIT_FIRST
```

以后，我们已经知道：

“我要打一个包。”

但是现在还没有真正建立这个 DIFI Packet。

因为我们还没有第一个 Sample。

我们冻结的接口规定：Data Packet 的时间戳属于**第一个 IQ Sample**，而不是 `pkt_start` 时刻。

DIFI_RTL_Interface_Spec_v1.0

所以这里需要等待：

```
纯文本
s_axis_iq_tvalid = 1
```

同时：

```
纯文本
time_valid = 1
```

并且我们自己：

```
纯文本
s_axis_iq_tready = 1
```

于是：

```
纯文本
TVALID && TREADY = 1
```

发生 handshake。

这个时钟沿发生很多重要事情。

你要在这一拍记住：

```
纯文本
```

```
IQ Sample0
Stream ID
samples_per_packet
time_sec
time_ps
```

因此：

```
纯文本
first_sample_reg
snapshot_stream_id
snapshot_samples
snapshot_time_sec
snapshot_time_ps
```

全部更新。

然后：

```
纯文本
header_index = 0
```

下一状态：

```
纯文本
SEND_HEADER
```

所以这一拍是 Packetizer 最重要的时钟沿之一。

第六课：为什么 Snapshot 必须存在?

假设：

```
纯文本
Stream ID = 1
N = 256
```

你正在发送这个包。

突然外面的配置寄存器变成：

```
Stream ID = 2
N = 128
```

如果你的 Header 或计数器一直直接读：

```verilog
cfg_data_stream_id
cfg_samples_per_packet
```

那一个 Packet 可能变成：

```
Word0 宣称 Size=263
Word1 Stream ID=1

发送到一半

配置改变

后面的逻辑突然认为 N=128
```

这显然不能接受。

所以：

```
cfg
  ↓
Packet 开始
  ↓
snapshot
  ↓
当前 Packet 永远只看 snapshot
```

因此：

```
cfg_samples_per_packet
```

和：

```
snapshot_samples
```

是完全不同的角色。

前者：

外部现在希望以后怎么配置。

后者：

当前正在发送的这个 Packet 已经确定使用什么配置。

第七课：SEND_HEADER 是最适合练计数器的地方

Header 一共有：

```
Word0
Word1
Word2
Word3
Word4
Word5
Word6
```

所以我们只需要一个：

```verilog
reg [2:0] header_index;
```

含义明确规定：

`header_index` = 当前正在输出第几个 Header Word。

于是：

```
header_index = 0 → Word0
header_index = 1 → Word1
header_index = 2 → Word2
```

```
...
header_index = 6 → Word6
```

组合逻辑中：

```verilog
case (header_index)

3'd0:
    // Header

3'd1:
    // Stream ID

3'd2:
    // OUI

...

endcase
```

这一步你以前写数电 FSM 时肯定见过，只不过这里状态内部又套了一个小计数器。

第八课：什么时候 header_index 才能 +1？

这就是 AXI4-Stream 的核心。

假设现在：

```
header_index = 2
```

所以：

```
TDATA = Word2
TVALID = 1
```

但：

```
```

```
TREADY = 0
```

下游在说：

“我现在接不了。”

那么这个周期：

```
纯文本
Word2 并没有真正传输。
```

所以：

```
纯文本
header_index 不能变成 3
```

下一拍仍然必须：

```
纯文本
TDATA = Word2
```

因此真正允许：

```
纯文本
header_index++
```

的条件只有：

```
纯文本
m_axis_pkt_tvalid && m_axis_pkt_tready
```

这个条件以后我们可以直接给它起一个心理名字：

```
纯文本
packet_output_fire
```

即：

输出 AXIS 真正完成一次传输。

你甚至可以在脑子里统一理解：

```
纯文本
fire = valid & ready
```

后面 AXI 用得多了，你看到：

```
纯文本
TVALID && TREADY
```

就应该自动想到：

这一拍数据真的过去了。

第九课：为什么组合逻辑就能保证 backpressure 时稳定?

这是一个很重要的问题。

假设：

```
纯文本
state = SEND_HEADER
header_index = 2
```

组合逻辑决定：

```
纯文本
TDATA = Word2
TVALID = 1
```

下游：

```
纯文本
TREADY = 0
```

由于没有 handshake：

```
纯文本
```

```
header_index 不变
state 不变
```

下一拍仍然：

```
纯文本
state = SEND_HEADER
header_index = 2
```

所以组合逻辑自然仍然输出：

```
纯文本
TDATA = Word2
TVALID = 1
```

这就是：

用“状态和计数器只在 handshake 时更新”来保证 AXIS 稳定性。

你不一定非要给每个 TDATA 再套一层寄存器。

第十课：Word6 发完后为什么不能直接进入普通 Payload?

因为 Sample0 已经提前被我们吃掉了。

回顾：

```
纯文本
WAIT_FIRST
```

时：

```
纯文本
Sample0 handshake
```

这个 Sample 已经从上游“消费”了。

只是暂存在：

```
纯文本
```

```
first_sample_reg
```

所以 Header 发完以后：

```
纯文本
Word6 handshake
```

下一状态首先应该：

```
纯文本
SEND_FIRST
```

发送：

```
纯文本
first_sample_reg
```

不能直接去读新的：

```
纯文本
s_axis_iq_tdata
```

否则 Sample0 就丢了。

这就是为什么我们的状态机里单独有：

```
纯文本
SEND_FIRST
```

第十一课：SEND_FIRST 有一个非常重要的边界情况

假如：

```
纯文本
N = 1
```

那么整个 Payload 就只有：

```
Sample0
```

所以：

```
SEND_FIRST
```

时已经是最后一个 Payload Word。

因此：

```
TLAST = 1
```

Sample0 handshake 后：

```
Packet 完成
seq + 1
pkt_done = 1
pkt_busy = 0
回 IDLE
```

不应该进入：

```
SEND_PAYLOAD
```

这就是为什么 RTL 设计里经常需要特别检查：

```
N=1
```

这种最小边界。

第十二课：N > 1 时才进入 SEND_PAYLOAD

例如：

```
N = 4
```

Sample0 已经发送了。

还需要：

```
Sample1
Sample2
Sample3
```

这时候 Packetizer 就可以近似“透明”了。

关系是：

```
IQ Input                    Packet Output

TDATA  --------------------> TDATA
TVALID --------------------> TVALID

TREADY <-------------------- TREADY
```

也就是：

```
输入 Sample 能不能被接受
=
输出 Sample 能不能被下游接受
```

所以：

```
s_axis_iq_tready ≈ m_axis_pkt_tready
```

这就是 backpressure 传播。

第十三课：payload_count 到底应该怎么定义？

这里非常重要。

RTL 中最怕的不是不会写，而是：

> **计数器语义模糊。**

例如你定义：

```verilog
reg [15:0] payload_count;
```

但脑子里没有一句明确的话说它代表什么。

那后面一定容易 off-by-one。

我建议我们明确规定：

> `payload_count` 表示“当前准备发送的 Payload Sample 的序号”。

Sample 从 0 开始：

```纯文本
Sample0
Sample1
Sample2
...
SampleN-1
```

但是 Sample0 在：

```纯文本
SEND_FIRST
```

单独处理。

所以进入：

```纯文本
SEND_PAYLOAD
```

时直接令：

```
payload_count = 1
```

然后：

```
payload_count = 1 → 当前 Sample1
payload_count = 2 → 当前 Sample2
...
payload_count = N-1 → 当前最后一个 Sample
```

于是最后 Sample 判断非常自然：

```
payload_count == snapshot_samples - 1
```

成立：

```
TLAST = 1
```

第十四课：以 N=4 完整跑一遍

这一步非常重要，我们用“时钟周期”的方式理解。

假设：

```
N = 4
Samples = A B C D
```

最理想情况下没有 backpressure。

第 1 段

```
```

```
state = IDLE
pkt_start = 1
```

时钟沿：

```
纯文本
state → WAIT_FIRST
pkt_busy → 1
```

第 2 段

```
纯文本
state = WAIT_FIRST
IQ=A
TVALID=1
time_valid=1
TREADY=1
```

发生 handshake。

时钟沿锁存：

```
纯文本
first_sample = A
N = 4
Stream ID
Timestamp
header_index = 0

state → SEND_HEADER
```

接下来输出

```
纯文本
Word0
```

handshake：

```
纯文本
```

```
header_index 0→1
```

下一拍：

```
‹/› 纯文本
Word1
```

……

一直：

```
‹/› 纯文本
Word6
```

handshake 后：

```
‹/› 纯文本
state → SEND_FIRST
```

SEND_FIRST

输出：

```
‹/› 纯文本
A
```

因为：

```
‹/› 纯文本
N=4 != 1
```

所以：

```
‹/› 纯文本
TLAST=0
```

A handshake 后：

```
payload_count = 1
state → SEND_PAYLOAD
```

SEND_PAYLOAD

当前：

```
payload_count=1
```

输出：

```
B
TLAST=0
```

handshake：

```
payload_count=2
```

然后：

```
C
```

handshake：

```
payload_count=3
```

现在：

```
3 == N-1
```

所以 D：

```
TDATA = D
TLAST = 1
```

D handshake 后：

```
data_seq++
pkt_done=1
pkt_busy=0
state=IDLE
```

一个 Packet 完成。

第十五课：再跑一次有 backpressure 的情况

假如正在发：

```
Word3
```

现在：

```
TVALID=1
TREADY=0
```

当前寄存器：

```
state = SEND_HEADER
header_index = 3
```

这一拍**什么都不更新**。

所以下一拍仍然：

```
state = SEND_HEADER
```

```
header_index = 3
```

所以：

```
TDATA仍然=Word3
TVALID仍然=1
```

再下一拍：

```
TREADY=1
```

终于：

```
TVALID && TREADY = 1
```

这时候：

```
header_index 3→4
```

下一拍才输出 Word4。

如果你真正理解了这个过程，AXI Stream 最核心的部分就恢复了。

第十六课：现在理解“时序 always”和“组合 always”分别负责什么

我们先不写完整代码，只写思想。

时序部分：

```verilog
always @(posedge aclk) begin

    if (!aresetn) begin

        state <= IDLE;
```

```
        ...

    end else begin

        case (state)

            IDLE:
                // pkt_start 时改变 state

            WAIT_FIRST:
                // handshake 时锁存各种 snapshot

            SEND_HEADER:
                // handshake 时 header_index++

            SEND_FIRST:
                // handshake 时决定完成或进入 payload

            SEND_PAYLOAD:
                // handshake 时 payload_count++

        endcase

    end
end
```

这里的核心词是：

更新

时序 always 负责：

```
状态更新
计数器更新
寄存器保存
```

组合部分：

```verilog
always @(*) begin

    // 默认输出
```

```
    case (state)

        IDLE:
            ...

        WAIT_FIRST:
            ...

        SEND_HEADER:
            ...

        SEND_FIRST:
            ...

        SEND_PAYLOAD:
            ...

    endcase
end
```

这里的关键词是：

当前应该输出什么

比如：

```
当前 SEND_HEADER + index=2
→ 输出 OUI
```

```
当前 SEND_FIRST
→ 输出 first_sample_reg
```

```
当前 SEND_PAYLOAD
→ 输出 s_axis_iq_tdata
```

把这两个职责分清以后，状态机就不会乱。

第十七课：为什么组合逻辑一定要先写默认值?

例如：

```verilog
always @(*) begin

    s_axis_iq_tready  = 1'b0;

    m_axis_pkt_tdata  = 32'd0;
    m_axis_pkt_tvalid = 1'b0;
    m_axis_pkt_tlast  = 1'b0;

    case (state)
        ...
    endcase
end
```

这是为了保证：

每个组合逻辑路径都给输出赋值。

否则比如你只在：

```纯文本
SEND_HEADER
```

状态写：

```verilog
m_axis_pkt_tvalid = 1'b1;
```

但在 IDLE 没写 `tvalid` 应该是什么。

综合工具就可能理解成：

“那我要保存上一拍的 tvalid。”

于是推断出 latch。

所以记住一个非常实用的习惯：

```
组合 always
    ↓
先给所有输出默认值
    ↓
case 里面覆盖需要改变的值
```

这会帮你避免很多 latch。

第十八课：现在不要急着写整个模块，我们分四步写

我建议你接下来实际动手时按照下面顺序。

第一步：只写“壳子”

先写：

```
module
所有 I/O
localparam 状态
内部 reg
endmodule
```

不要写任何 always。

你先确认：

> 这个模块有哪些“记忆”。

第二步：只写状态机骨架

先不产生正确 DIFI Word。

让：

```
IDLE
→ WAIT_FIRST
→ SEND_HEADER
→ SEND_FIRST
```

```
→ SEND_PAYLOAD
→ IDLE
```

跑起来。

甚至：

```
m_axis_pkt_tdata
```

暂时全部输出 0 都可以。

目的只是：

> 状态转移正确。

第三步：再填 Word0~Word6

等状态机骨架正确以后，再实现：

```
header_index
```

让它输出：

```
Word0
...
Word6
```

第四步：最后接 Payload

再处理：

```
first_sample_reg
```

以及：

```
SEND_PAYLOAD
```

最后才加：

```
TLAST
Sequence
pkt_done
```

不要第一次就同时解决所有问题。

我们现在正式开始第一步

你现在先自己写 **模块骨架**，先不要写 `always`。

按照文档第 22 页冻结的接口，应该包含这些端口：`aclk/aresetn`、`pkt_start/pkt_busy/pkt_done`、IQ AXIS、配置、Time Source、Packet AXIS。

DIFI_RTL_Interface_Spec_v1.0

然后内部你先定义这些东西：

```
5 个 state localparam
state

data_seq

first_sample_reg

snapshot_stream_id
snapshot_samples
snapshot_time_sec
snapshot_time_ps

header_index
payload_count
```

你可以自己决定具体位宽，但需要能解释为什么。

这一步你自己写。

把你写出来的 `module ... endmodule` 骨架发给我，不需要写 `always`。我下一步会逐行帮你检查，然后我们只写 **IDLE → WAIT_FIRST** 这一小段状态机，不会一下跳到完整 Packetizer。这样你会真正重新把 RTL 的感觉找回来，而不是照着成品代码抄。

你

我有一个问题，如果按照“rtl/
├── include/
│ └── difi_v1_1_defs.vh
└── data/
└── difi_data_packetizer.v”这种目录结构，difi_data_packetizer.v 的头部需要怎么把 difi_v1_1_defs.vh include进来?

ChatGPT

可以。按照我们现在约定的目录：

```
rtl/
├── include/
│   └── difi_v1_1_defs.vh
└── data/
    └── difi_data_packetizer.v
```

我更推荐的做法不是在 `.v` 里写相对路径，而是**把 `rtl/include` 配置成 Vivado 的 Verilog Include Directory**，然后在 `difi_data_packetizer.v` 顶部直接写：

```verilog
`timescale 1ns / 1ps

`include "difi_v1_1_defs.vh"

module difi_data_packetizer (
    ...
);
```

这也是我们接口规格里推荐“公共常量放到 `rtl/include/difi_v1_1_defs.vh`”的目的。

DIFI_RTL_Interface_Spec_v1.0

为什么不推荐直接写 `../include/`

你确实可以尝试：

```verilog
`include "../include/difi_v1_1_defs.vh"
```

但我不推荐这么做，因为这样会让源文件和目录结构强耦合。以后如果变成：

```
rtl/
├── include/
├── data/
├── context/
├── version/
└── common/
```

不同目录里的文件就会到处出现：

```verilog
`include "../include/..."
```

甚至目录层级一变就全部失效。

更工程化的方式是：

```
告诉 Vivado：
“以后找 .vh，就去 rtl/include 这个目录找。”
```

于是所有模块统一写：

```verilog
`include "difi_v1_1_defs.vh"
```

这样最好维护。

在 Vivado 中怎么设置

如果你的工程目录比如：

```
project/
├── rtl/
│   ├── include/
│   │   └── difi_v1_1_defs.vh
│   └── data/
│       └── difi_data_packetizer.v
```

首先把：

```
difi_v1_1_defs.vh
```

加入 Vivado 工程，文件类型设为 **Verilog Header**。

然后给 `sources_1` 设置 Include Directory 为：

```
<你的工程路径>/rtl/include
```

也可以直接在 Tcl Console 中设置，例如：

```tcl
set_property include_dirs {D:/your_project/rtl/include} [get_filesets sources_1]
```

如果之后 testbench 也要：

```verilog
`include "difi_v1_1_defs.vh"
```

那么仿真 fileset 最好也加入：

```tcl
set_property include_dirs {D:/your_project/rtl/include} [get_filesets sim_1]
```

Windows 下在 Vivado/Tcl 路径里建议用：

```
/
```

而不是：

```
\
```

`.vh` 到底是什么？

这一点也建议现在顺便弄清楚。

假如 `difi_v1_1_defs.vh` 里面有：

```verilog
`define DIFI_DATA_PKT_TYPE       4'h1
`define DIFI_OUI_WORD            32'h006A_621E
`define DIFI_DATA_CLASS_WORD     32'h0000_0000
```

然后 Packetizer：

```verilog
`include "difi_v1_1_defs.vh"

module difi_data_packetizer (...);

    ...

    m_axis_pkt_tdata = `DIFI_OUI_WORD;

endmodule
```

预处理器实际上可以近似理解为，在正式编译 Verilog 之前，把：

```verilog
`include "difi_v1_1_defs.vh"
```

的位置直接展开成头文件里的文本。

所以它和：

```verilog
difi_xxx u_xxx (...);
```

这种**模块实例化完全不是一回事。**

`.vh` 里一般放的是：

```
`define 宏
公共常量
条件编译定义
```

而不是生成一个硬件模块。

我们这个项目建议统一成这样

所有需要 DIFI 固定常量的 RTL：

```
difi_data_packetizer.v
difi_data_parser.v
difi_context_generator.v
difi_context_parser.v
difi_version_generator.v
...
```

头部统一：

```verilog
`include "difi_v1_1_defs.vh"
```

以后协议常量比如 OUI 从一个版本变到另一个版本，只改：

```
difi_v1_1_defs.vh
```

而不用去十几个 `.v` 文件里搜 `32'h006A621E`。

所以你现在创建工程时，就建议采用“配置 Include Directory + 文件中只写文件名”的方式。 这样我们接下来写 `difi_data_packetizer.v` 时，头部就很干净。

你

我有个问题，tready信号应该放到时序逻辑还是组合逻辑来控制?

ChatGPT

都可以，但**不能只看“信号类型”，要看你有没有能力保证握手语义。**

对我们这个 `difi_data_packetizer`，我建议你先这样理解：

`TREADY` 表示“这一拍如果上游给我一个有效数据，我能不能在这个时钟沿把它接下来”。

因此它既可以由组合逻辑产生，也可以由寄存器产生；区别主要在于**时序路径和是否需要缓冲。**

我们冻结的接口规范只规定了：`s_axis_iq_tready` 用来对上游施加 backpressure，并且只有 `TVALID && TREADY` 才算真正接收一个 Sample；并没有规定它必须组合实现还是时序实现。

DIFI_RTL_Interface_Spec_v1.0

先看最容易理解的组合逻辑写法

例如在 `WAIT_FIRST`：

verilog

```verilog
always @(*) begin
    s_axis_iq_tready = 1'b0;

    case (state)
        ST_WAIT_FIRST: begin
            if (time_valid)
                s_axis_iq_tready = 1'b1;
        end
    endcase
end
```

它表达的意思非常直接：

```
当前在 WAIT_FIRST
并且 time_valid=1
        ↓
我现在就能接 IQ
        ↓
TREADY=1
```

然后真正接收数据仍然是在时序逻辑中判断：

```verilog
always @(posedge aclk) begin
    if (s_axis_iq_tvalid && s_axis_iq_tready) begin
        first_sample_reg <= s_axis_iq_tdata;
    end
end
```

注意这里的分工：

```
组合逻辑：
决定“现在能不能接”

时序逻辑：
在时钟沿真正完成接收并保存
```

这是非常自然的一种写法。

对你这个 Packetizer，`WAIT_FIRST` 阶段我推荐组合产生 TREADY

因为是否能接受首 Sample，本质取决于当前状态和当前条件：

```
state == WAIT_FIRST
time_valid == 1
配置合法
```

所以完全可以写成类似：

```verilog
```

```
s_axis_iq_tready =
    (state == ST_WAIT_FIRST) &&
    time_valid;
```

这很好理解，也很适合作为第一版。

真正比较微妙的是 SEND_PAYLOAD

我们前面说过，最简单的思路是：

```
IQ Source → Packetizer → Packet Output
```

在 Payload 阶段近似直接透传：

```verilog
m_axis_pkt_tdata  = s_axis_iq_tdata;
m_axis_pkt_tvalid = s_axis_iq_tvalid;

s_axis_iq_tready  = m_axis_pkt_tready;
```

也就是：

```
下游能接
   ↓
Packetizer 能把这个 Sample 发出去
   ↓
Packetizer 就允许上游把 Sample 给我
```

这在逻辑上非常直观。

例如：

```
m_axis_pkt_tready = 0
        ↓
s_axis_iq_tready = 0
```

```
        ↓
上游保持当前 Sample
```

所以数据不会丢。

但是它有一个工程问题

你形成了这样一条组合路径：

```
下游 m_axis_pkt_tready
        ↓
组合逻辑
        ↓
上游 s_axis_iq_tready
```

也就是说，下游的 `READY` 可以一路组合传播到上游。

如果将来：

```
模块A → 模块B → 模块C → 模块D
```

每一级都这么写：

```
s_ready = m_ready
```

最后可能形成非常长的：

```
READY combinational chain
```

导致时序收敛困难。

更严格的 AXI 工程实现通常会用**寄存器切断 READY 路径**，或者使用 skid buffer / FIFO。

那我是不是直接把 TREADY 放到时序 always 里就行?

这里恰恰是容易踩坑的地方。

假设你简单写：

```verilog
always @(posedge aclk) begin
    if (m_axis_pkt_tready)
        s_axis_iq_tready <= 1'b1;
    else
        s_axis_iq_tready <= 1'b0;
end
```

乍看是在“寄存 TREADY”。

但假如：

```
第 n 拍：

s_axis_iq_tready = 1
m_axis_pkt_tready = 1
```

然后下游突然：

```
第 n+1 拍：

m_axis_pkt_tready = 0
```

因为 `s_axis_iq_tready` 是寄存器，它可能直到时钟沿以后才变为 0。

那么这个周期可能出现：

```
上游看到 TREADY=1
        ↓
认为一个新的 Sample 被 Packetizer 接收
```

但：

```
下游 TREADY=0
```

Packetizer 又没地方保存它。

结果：

> **你接受了一个 Sample，却没法把它发送出去。**

数据就可能丢了。

所以：

> **把 TREADY 寄存起来，通常意味着你必须有内部缓冲。**

比如：

```
                +-----------+
IQ Source ---> | 1-word    | ---> Packet Output
               | Buffer    |
               +-----------+
```

哪怕下游突然停下来：

```
m_axis_pkt_tready = 0
```

你已经接受的那个 Sample 还能暂存在 Buffer 中。

这就是 skid buffer / elastic buffer 的基本思想。

所以对于我们第一版，我推荐什么？

我建议你现在**不要为了“所有 READY 必须寄存”而增加复杂度。**

第一版按我们目前的学习目标：

`WAIT_FIRST`

组合逻辑控制：

```
WAIT_FIRST
+
time_valid
```

```
+
允许建立 Packet
        ↓
s_axis_iq_tready = 1
```

SEND_HEADER

```
纯文本
s_axis_iq_tready = 0
```

因为第一版 Header 输出期间不继续接后面的 IQ。这个行为也是我们接口规格明确允许的。

DIFI_RTL_Interface_Spec_v1.0

SEND_FIRST

仍然：

```
纯文本
s_axis_iq_tready = 0
```

因为现在是在发送之前缓存的 Sample0。

SEND_PAYLOAD

第一版为了容易理解，可以使用组合 backpressure：

```verilog
s_axis_iq_tready = m_axis_pkt_tready;
```

同时：

```verilog
m_axis_pkt_tvalid = s_axis_iq_tvalid;
m_axis_pkt_tdata  = s_axis_iq_tdata;
```

这样你能非常直观地理解 AXIS handshake。

等这个版本仿真正确以后，我们以后做吞吐/时序优化，再把这里替换成：

```
纯文本
```

```
1-deep elastic buffer
```

或者：

```
small FIFO
```

来切断组合 READY 路径。

一个非常重要的区分

你现在不要把：

```
“TREADY 是组合信号”
```

理解成：

> 数据是在组合逻辑里被“接收”的。

不是。

比如：

```verilog
assign s_axis_iq_tready = ...;
```

只是告诉上游：

> “当前这个周期，我有能力接。”

真正的数据传输事件还是：

```
posedge aclk
+
TVALID=1
+
TREADY=1
```

即：

```
组合逻辑产生条件
        ↓
时钟沿完成事务
```

这个理解很重要。

你可以把 AXIS 想成这样

`TVALID` 是发送者说：

“我手里有东西。”

`TREADY` 是接收者说：

“我现在有地方放。”

但东西真正交接，是两个人在**同一个时钟沿**同时说：

```
VALID = 1
READY = 1
```

于是：

```
transfer = TVALID && TREADY
```

所以后面你写 RTL 时，可以经常定义心理上的两个事件：

```
iq_fire  = s_axis_iq_tvalid && s_axis_iq_tready

pkt_fire = m_axis_pkt_tvalid && m_axis_pkt_tready
```

然后时序逻辑里尽量围绕：

```
if (iq_fire)
```

和：

```
纯文本
if (pkt_fire)
```

来更新状态和计数器。

这会让你的代码清晰很多。

你这个 Packetizer 第一版可以先遵循这条原则

```
纯文本
TREADY / TVALID / TDATA / TLAST：
主要由组合逻辑根据 state 产生

state / counter / snapshot / seq：
全部在时序逻辑中更新
```

这不是唯一正确写法，但非常适合你现在重新熟悉 RTL。

等我们把这个模块第一版写通，再专门回头看“为什么工业 AXI 设计经常注册 READY、skid buffer 怎么实现”，你会容易理解得多。

你

在我们商定好的端口设计中，同样是外部配置信息，为什么time相关变量有valid信号？但是cfg_data_stream_id、和cfg_samples_per_packet却没有？module difi_data_packetizer (
    input wire   aclk,
    input wire   aresetn,

    input wire   pkt_start,
    output reg   pkt_busy,
    output reg   pkt_done,

    input wire [31:0] s_axis_iq_tdata,
    input wire   s_axis_iq_tvalid,
    output reg   s_axis_iq_tready,

    input wire [31:0] cfg_data_stream_id,

```
    input  wire [15:0] cfg_samples_per_packet,

    input  wire [31:0] time_sec,
    input  wire [63:0] time_ps,
    input  wire        time_valid,

    output reg  [31:0] m_axis_pkt_tdata,
    output reg         m_axis_pkt_tvalid,
    input  wire        m_axis_pkt_tready,
    output wire [3:0]  m_axis_pkt_tkeep,
    output reg         m_axis_pkt_tlast
);
...
```

显示更多

ChatGPT

这个区别其实很关键。虽然它们看起来都是“外部输入”，但在我们冻结的接口里，**`cfg_*` 和 `time_*` 的角色并不一样。**

`cfg_data_stream_id`、`cfg_samples_per_packet` 属于**持续存在的 active configuration**；而 `time_sec/time_ps` 属于**随时间实时变化的时间源输出。**接口规格里也把它们明确分成了 Config 和 Time 两组。 DIFI_RTL_Interface_Spec_v1.0

`cfg_*` 为什么不需要 `valid`

我们对配置的假设是：

> 只要 Packetizer 被允许启动，`cfg_data_stream_id` 和 `cfg_samples_per_packet` 就代表“当前已经生效的配置”。

也就是说，它们更像寄存器输出：

```
纯文本
配置寄存器
    |
    ├── cfg_data_stream_id
    └── cfg_samples_per_packet
            ↓
      difi_data_packetizer
```

它们不是“一拍一拍到来的数据事务”，而是一个**持续保持的状态值。**

比如：

```
cfg_data_stream_id       = 32'h00000001
cfg_samples_per_packet   = 16'd256
```

可能连续保持几百万个时钟周期都不变。

Packetizer 在首 Sample handshake 的时候，只需要做：

```
snapshot_stream_id = cfg_data_stream_id
snapshot_samples   = cfg_samples_per_packet
```

然后当前 Packet 就使用这份 snapshot。文档正是这样规定的。 DIFI_RTL_Interface_Spec_v1.0

所以这里默认：

```
cfg_* 一直“有意义”
```

而不是：

```
某一拍才突然有效
```

因此没有额外的 `cfg_valid`。

但时间源不一样

`time_sec/time_ps` 更像：

```
“当前这一刻系统认为现在是什么时间”
```

而且这个时间源不一定始终可信。

例如以后你的 Time Source 可能经历：

```
刚复位
↓
PS 还没写入 UTC 基准
↓
PL 本地计时器还没初始化
↓
此时 time_sec/time_ps 虽然线上有二进制值，
但这些值没有时间意义
```

过一会：

```
时间源初始化完成
↓
time_valid = 1
```

这时：

```
time_sec
time_ps
```

才可以拿来写进 DIFI Timestamp。

我们的协议规格因此明确规定：Packetizer 不负责校时，只消费统一 Time Source 提供的 `time_sec/time_ps/time_valid`；Data Packet 必须在接受第一个 IQ Sample 时锁存时间，而且 `time_valid=0` 时不能接受这个 Packet 的第一个 Sample。 DIFI_1.1_协议实现规格_v1.0

所以 `time_valid` 实际上是在说：

> **“现在这组时间数值能不能被解释为合法的 Packet Timestamp？”**

可以把两组接口类比成不同东西

配置更像一个控制面寄存器：

```
DATA_STREAM_ID register = 1
SAMPLES_PER_PACKET register = 256
```

它们一旦被 commit，就一直在那里。

时间源则更像一个传感器：

```
纯文本
time = 13:54:21.123...
```

但这个传感器可能还没有初始化完成，所以必须另外告诉你：

```
纯文本
这个读数现在可靠吗？
```

于是就有：

```
纯文本
time_valid
```

那配置真的完全不需要“有效性”吗？

这里要区分两种 valid。

一种是我们现在没有的：

```
纯文本
cfg_valid
```

表示：

> 整套配置是否已经准备好。

另一种是：

```
纯文本
cfg_samples_per_packet 是否合法
```

例如：

```
纯文本
0
257
```

```
65535
```

这些显然不合法，因为我们项目冻结范围是：

```
1 ~ 256
```

DIFI_1.1_协议实现规格_v1.0

但我们当前的接口设计选择是：

> **Packetizer 不负责通过一个 `cfg_valid` 输入判断配置是否已经初始化。**

未来真正有：

```
AXI4-Lite
    ↓
shadow registers
    ↓
cfg_commit
    ↓
active registers
```

以后，Packetizer 看到的：

```
cfg_data_stream_id
cfg_samples_per_packet
```

应该已经来自：

```
cfg_active
```

即"已经提交并生效"的配置。协议规格里的配置模型就是 `shadow → active` 原子提交。

DIFI_1.1_协议实现规格_v1.0

所以系统最终会更像：

```
```

```
          AXI4-Lite
              |
              v
       +---------------+
       | difi_cfg_regs |
       |               |
       | shadow regs   |
       |      ↓ commit |
       | active regs   |
       +-------+-------+
               |
     cfg_data_stream_id
     cfg_samples_per_packet
               |
               v
      difi_data_packetizer
```

Packetizer 默认相信：

> “你给我的 active config 已经是系统认可的配置。”

这属于模块职责分层。

为什么不用同样的方法对时间做一个“active time register”？

因为配置和时间还有一个根本区别：

配置是**离散变化的状态**：

```
纯文本
N = 256
保持……
保持……
保持……
用户修改
↓
N = 128
```

时间却是**连续运行的数据**：

```
纯文本
100.000000001 s
100.000000002 s
100.000000003 s
...
```

所以不能简单把时间理解成“配置寄存器”。

Packetizer需要的是：

> 第一个 Sample 到来的那个时刻，时间源当前的瞬时值。

因此才是在首 Sample handshake 当拍：

```
snapshot_time_sec <= time_sec
snapshot_time_ps  <= time_ps
```

而不是在 `pkt_start` 时就锁存。

还有一个容易混淆的点

`time_valid` 并不表示：

> “时间刚更新了一次。”

它不是类似 AXI Stream 的 `TVALID`。

它更接近一个**状态位**：

```
time_valid = 1
```

表示：

> 当前 `time_sec/time_ps` 是可用的、具有正确时间语义的。

所以它可能连续保持：

```
1 1 1 1 1 1 1 1 ...
```

同时：

```
time_sec/time_ps
```

不断变化。

这和：

```
纯文本
s_axis_iq_tvalid
```

的事务意义并不一样。

如果我们硬要给配置也加 `cfg_valid` 可以吗？

当然可以，设计上并没有违反什么。

比如：

```verilog
input wire cfg_valid;
```

然后：

```
纯文本
pkt_start
↓
必须 cfg_valid=1
↓
才允许建立 Packet
```

但是我们当前冻结的模块没有这样设计，因为它会引入一个职责问题：

> 到底是谁负责判断“配置已经完成初始化并合法”？

我们目前把这个责任放在未来的：

```
纯文本
difi_cfg_regs / system control
```

而不是 Data Packetizer。

这样 Packetizer 会简单很多：

```
Packetizer负责：
“拿 active config 来打包。”

而不是：
“管理配置生命周期。”
```

这也符合我们之前冻结的“Packetizer 不负责 AXI-Lite 地址译码和配置管理”的模块边界。

DIFI_RTL_Interface_Spec_v1.0

所以你可以这样记

```
cfg_*：
“当前系统配置是什么？”
→ 默认始终存在
→ 在 Packet 开始时 snapshot

time_*：
“现在是什么时间？”
→ 数值一直变化
→ 但不一定始终可信
→ 所以需要 time_valid
→ 在第一个 Sample handshake 时 snapshot
```

这也是为什么这三个时间信号是一组：

```verilog
time_sec
time_ps
time_valid
```

而配置接口只是：

```verilog
cfg_data_stream_id
cfg_samples_per_packet
```

这不是“不对称设计”，而是因为它们代表的是两类完全不同的上游信息。

你

对于sample 0和时间数据，在设计时它们是同时snapshot吗？还是说一个根据axi握手时机snapshot一个根据valid信号snapshot？stream ID和samples_per_packet 呢？它们随前两种信号的哪个一起snapshot?

ChatGPT

对，按照我们已经冻结的接口语义，**Sample0、时间、Stream ID、samples_per_packet 四者应该在同一个事件、同一个时钟沿完成 snapshot。**

不是"Sample0 按 AXI 握手 snapshot，时间按 `time_valid` 单独 snapshot"。`time_valid` 在这里是**准入条件**，不是独立的"采样触发信号"。接口规格明确规定：只有当 `time_valid=1` 且首个 IQ Sample 发生 `TVALID && TREADY` 握手时，才正式建立这个 Packet；同一时钟沿锁存 first sample、时间和配置。 DIFI_RTL_Interface_Spec_v1.0

可以把真正的 snapshot 事件理解成：

```verilog
first_sample_fire =
    s_axis_iq_tvalid &&
    s_axis_iq_tready;
```

而我们在 `WAIT_FIRST` 状态中又会要求：

```
time_valid = 1
```

才允许：

```
s_axis_iq_tready = 1
```

所以实际效果等价于：

```
当前状态 = WAIT_FIRST
AND
time_valid = 1
```

```
AND
s_axis_iq_tvalid = 1
AND
s_axis_iq_tready = 1
            ↓
     首 Sample 真正被接受
            ↓
     同一时钟沿 snapshot：
       Sample0
       time_sec
       time_ps
       cfg_data_stream_id
       cfg_samples_per_packet
```

`time_valid` 本身不会触发 snapshot

比如现在：

```
time_valid = 1
```

但是：

```
s_axis_iq_tvalid = 0
```

说明时间源已经可靠，但第一个 Sample 还没来。

这时候：

```
time_sec/time_ps
```

可以每拍继续变化，我们**一个都不保存**。

比如：

```
cycle 1: time = 100.001 s, time_valid=1, IQ valid=0
cycle 2: time = 100.002 s, time_valid=1, IQ valid=0
cycle 3: time = 100.003 s, time_valid=1, IQ valid=1
```

如果 cycle 3 发生 IQ handshake，那么锁存的是：

```
100.003 s
```

而不是 `time_valid` 第一次变成 1 时的：

```
100.001 s
```

因为协议语义是：

> **Data Packet Timestamp 表示该 Packet 第一个 IQ Sample 的时间。**

DIFI_1.1_协议实现规格_v1.0

反过来也一样

假设 Sample0 很早就来了：

```
s_axis_iq_tvalid = 1
```

但是：

```
time_valid = 0
```

那么我们当前设计下：

```
s_axis_iq_tready = 0
```

所以：

```
TVALID=1
TREADY=0
```

并没有发生 handshake。

于是：

```
Sample0 不会被 Packetizer 消费
时间也不会 snapshot
配置也不会 snapshot
```

上游必须继续保持这个 Sample。

等某一拍：

```
time_valid = 1
```

Packetizer 拉高：

```
TREADY=1
```

于是那个时钟沿才真正：

```
Sample0 + 时间 + 配置
一起锁存
```

那 Stream ID 和 samples_per_packet 为什么也跟 Sample0 一起锁存?

因为我们需要一个非常明确的 **Packet 建立点。**

这个建立点就是：

> **第一个 IQ Sample 被正式接受的那个时钟沿。**

如果配置在 `pkt_start` 时 snapshot，而时间和 Sample0 在之后 snapshot，就可能出现这样的情况：

```
t0:
pkt_start
```

```
cfg Stream ID = 1
N = 256
→ 配置 snapshot

t1:
外部配置更新
Stream ID = 2
N = 128

t2:
Sample0 终于来了
→ Sample0/time snapshot
```

那么当前包到底应该被认为属于：

```
Stream ID = 1
N = 256
```

还是：

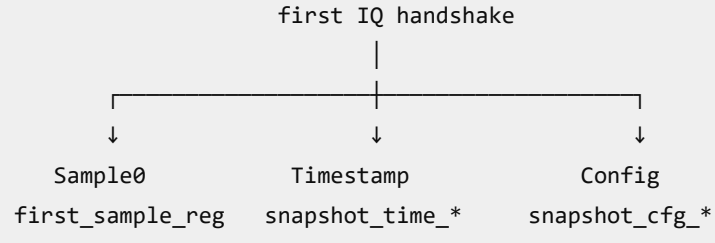
```
Stream ID = 2
N = 128
```

虽然我们也可以人为规定一种答案，但这样会让 Packet 的事务边界变得不统一。

所以我们冻结的是更简单、清楚的规则：

当前 Packet 的所有动态属性，都以“首 Sample handshake”这个事件为统一 snapshot 点。

也就是：



这也是接口文档第 4.3 节明确写下的行为。 DIFI_RTL_Interface_Spec_v1.0

`pkt_start` 到底干了什么?

它只是:

创建一次"准备发 Data Packet"的事务。

可以理解为:

```
pkt_start
   ↓
IDLE → WAIT_FIRST
   ↓
"我已经准备好建立一个 Packet 了"
```

但这个时候 Packet 的具体内容还没有冻结。

真正冻结发生在:

```
首 Sample handshake
```

所以三者时间关系是:

```
pkt_start
    │
    │  可能等几拍
    ▼
WAIT_FIRST
    │
    │ time_valid=1
    │ IQ valid=1
    ▼
first sample handshake
    │
    ├── snapshot Sample0
    ├── snapshot time_sec/time_ps
    ├── snapshot Stream ID
    └── snapshot samples_per_packet
    │
    ▼
SEND_HEADER
```

写 RTL 时你可以把这个事件集中写在一个地方

不用现在照抄代码，但结构上最好像这样：

```verilog
ST_WAIT_FIRST: begin
    if (s_axis_iq_tvalid && s_axis_iq_tready) begin

        // 同一个 Packet 建立事件

        first_sample_reg   <= s_axis_iq_tdata;

        snapshot_time_sec  <= time_sec;
        snapshot_time_ps   <= time_ps;

        snapshot_stream_id <= cfg_data_stream_id;
        snapshot_samples   <= cfg_samples_per_packet;

        ...
    end
end
```

而不要写成：

```verilog
if (time_valid)
    snapshot_time <= time;

if (s_axis_iq_tvalid && s_axis_iq_tready)
    first_sample <= ...;
```

后者会让时间和 Sample0 很可能来自不同的时钟周期，这正是我们不希望发生的。

最终你可以记成一句话

`time_valid` 决定“现在能不能建立 Packet”，IQ handshake 决定“Packet 在什么时候真正建立”；一旦建立，Sample0、Timestamp、Stream ID、samples_per_packet 全部在同一个上升沿 snapshot。

这四者之后直到当前 Packet 完成，都只使用 snapshot 值，不再看外部实时输入。

你

是不是只有s_axis_iq_tready有效，上游才会推进数据？因此在snapshot之前，s_axis_iq_tready是不是必须保持一定时钟周期？

ChatGPT

不是“`TREADY=1` 上游就推进”，而是：

> **只有 `TVALID && TREADY == 1` 的那个有效时钟沿，上游和下游才同时把当前这一个 Sample 视为已经传输完成。**

这正是我们接口规范里冻结的 AXI4-Stream 推进条件。 DIFI_RTL_Interface_Spec_v1.0

所以对 IQ 输入：

```
纯文本
s_axis_iq_tvalid = 1   上游说：“当前 Sample 有效”
s_axis_iq_tready = 1   Packetizer 说：“我现在能接”
```

两者同时为 1，到了上升沿：

```
纯文本
posedge aclk
    ↓
Sample handshake
    ↓
上游可以推进到下一个 Sample
Packetizer snapshot 当前 Sample
```

1. `TREADY=1` 本身不会让上游推进

比如：

```
纯文本
TVALID = 0
TREADY = 1
```

只是 Packetizer 在说：

“我已经准备好了，你有数据就可以给我。”

但现在没有有效 Sample，所以没有任何数据传输。

反过来：

```
TVALID = 1
TREADY = 0
```

表示：

> 上游有 Sample，但 Packetizer 现在不接。

此时上游必须保持：

```
TVALID = 1
TDATA  = 当前这个 Sample
```

不能擅自切换到下一个 Sample。

真正推进只有：

```
TVALID = 1
TREADY = 1
         ↓
      上升沿
         ↓
     handshake
```

---

2. 那 `TREADY` 要不要提前保持若干个时钟周期？

不需要。

这是你这个问题最关键的答案。

AXI4-Stream 没有要求：

> `TREADY` 必须提前 1 拍、2 拍，或者保持若干周期以后才能接数据。

只要求在某个采样上升沿到来时：

```
```

```
TVALID = 1
TREADY = 1
```

并满足正常同步电路的 setup/hold 时序要求，那么**这一拍就可以完成 handshake**。

例如：

```
纯文本
```

```
         cycle n                cycle n+1

clk      ↑----------------------↑

TVALID   1======================1
TDATA    Sample0=================Sample0

TREADY   0---------1=============1
                            ↑
                       这里已经稳定

                              下一个 ↑
                              handshake
```

在第二个上升沿：

```
纯文本
```

```
TVALID=1
TREADY=1
```

所以直接接收 Sample0。

不需要再多等一拍。

---

3. 对我们的 Packetizer，snapshot 就发生在 handshake 的那个沿

假设当前：

```
纯文本
```

```
state = WAIT_FIRST
```

并且时间源有效：

```
纯文本
```

```
time_valid = 1
```

于是 Packetizer 可以组合地产生：

```
s_axis_iq_tready = 1
```

如果这时候上游已经：

```
s_axis_iq_tvalid = 1
s_axis_iq_tdata  = Sample0
```

那么在接下来的上升沿：

```
s_axis_iq_tvalid && s_axis_iq_tready
                 =
                 1
```

于是**就在这个上升沿同时 snapshot**：

```
Sample0
time_sec
time_ps
cfg_data_stream_id
cfg_samples_per_packet
```

不需要：

```
先 TREADY=1
等一拍
再 snapshot
```

我们冻结的接口语义正是“首 Sample handshake 同一时钟沿完成 Sample、时间和配置 snapshot”。 DIFI_RTL_Interface_Spec_v1.0

---

4. 可以用一个实际时序例子理解

假设进入 `WAIT_FIRST` 后：

```
纯文本
Cycle 0:
state = WAIT_FIRST
time_valid = 0
TREADY = 0
```

上游已经有数据：

```
纯文本
TVALID = 1
TDATA  = Sample0
```

但是：

```
纯文本
TREADY = 0
```

所以 Sample0 没被接收。

上游必须保持 Sample0。

然后时间源变有效：

```
纯文本
Cycle 1:
time_valid = 1
```

你的组合逻辑马上得到：

```
纯文本
TREADY = 1
```

现在在 Cycle 1 末尾的那个上升沿：

```
纯文本
TVALID = 1
TREADY = 1
```

于是：

```
纯文本
handshake!
```

同一个沿：

```
纯文本
first_sample_reg     <= Sample0
snapshot_time_sec    <= time_sec
snapshot_time_ps     <= time_ps
snapshot_stream_id   <= cfg_data_stream_id
snapshot_samples     <= cfg_samples_per_packet
```

然后上游在**这个 handshake 之后**才可以准备 Sample1。

所以整个逻辑是：

```
纯文本
Sample0 一直等着
        │
        │ TREADY突然变1也没问题
        ▼
下一个上升沿 handshake
        │
        ├─ Packetizer锁存Sample0
        │
        └─ 上游允许推进到Sample1
```

---

5. 这里还有一个 AXI 很重要的原则

作为 Source 的上游，理论上不应该这样写：

```
纯文本
只有看到 TREADY=1
我才把 TVALID 拉高
```

更规范的行为是：

上游一旦有数据，就可以拉高 `TVALID`，不需要等待 `TREADY`。

然后如果 Packetizer：

```
TREADY=0
```

上游就保持：

```
TVALID=1
TDATA不变
```

直到 handshake。

也就是：

```
Source:
“我有数据，就声明 VALID。”

Sink:
“我有空间，就声明 READY。”

两边独立决定。
```

这样可以避免握手双方都互相等待。

---

6. 为什么我们可以放心在 handshake 当拍 snapshot?

因为在同步设计里，时钟沿之前：

```
TDATA
TVALID
TREADY
time_sec
time_ps
cfg_*
```

都应该已经满足 setup time。

到：

```
```

```
posedge aclk
```

寄存器同时采样这些输入。

所以 RTL 中：

```verilog
if (s_axis_iq_tvalid && s_axis_iq_tready) begin
    first_sample_reg   <= s_axis_iq_tdata;
    snapshot_time_sec  <= time_sec;
    snapshot_time_ps   <= time_ps;
    ...
end
```

完全成立。

你不需要人工规定：

- `TREADY` 先保持 2 个周期。

setup/hold 是否满足是综合实现后的**时序分析**负责检查的，不是靠你人为多等几个 cycle 来满足。

---

7. 但有一个前提：这些信号必须在同一时钟域

我们的第一版接口已经假定 Packetizer 使用单一 `aclk` 时钟域，CDC 不在模块内部处理。

DIFI_RTL_Interface_Spec_v1.0

所以我们默认：

```
s_axis_iq_*
time_*
cfg_*
```

在到达 Packetizer 前，都已经满足 `aclk` 时钟域的时序要求。

如果未来：

```
ADC 时钟域
    ↓
```

```
Packetizer aclk 时钟域
```

那就不能直接这么接，需要：

```
纯文本
Async FIFO / CDC
```

这是另一个问题。

你现在最应该建立的一个概念

不要把：

```
纯文本
TREADY=1
```

理解成：

> “已经接受数据了。”

它只表示：

> **“如果这个周期 TVALID 也为 1，那么下一个有效时钟沿我就会接受它。”**

真正的数据所有权转移点永远是：

```
纯文本
posedge aclk
且
TVALID && TREADY
```

因此对于我们现在的首 Sample，**不需要让 `s_axis_iq_tready` 提前保持若干周期。只要它在握手那个上升沿之前稳定为 1，就可以在那个沿直接 snapshot Sample0、Timestamp 和 cfg snapshot。**

你

m_axis_pkt_tvalid建议放到组合逻辑块还是时序逻辑块?

ChatGPT

对我们现在这一版 `difi_data_packetizer`，我建议 **`m_axis_pkt_tvalid` 放在组合逻辑块里产生**，和 `m_axis_pkt_tdata / tlast / s_axis_iq_tready` 一起由当前 `state` 决定。

原因不是“`TVALID` 天生就该组合”，而是我们当前这版结构非常适合这样写：

```
纯文本
state / counter / snapshot
        ↓
   组合逻辑决定
        ↓
TDATA / TVALID / TLAST / TREADY
```

而：

```
纯文本
state / counter / snapshot
```

只在时钟沿更新。

这和我们之前冻结的“只有 `TVALID && TREADY` 才推进状态和计数器”的接口规则正好配合。
DIFI_RTL_Interface_Spec_v1.0

比如你在 `SEND_HEADER`：

```verilog
always @(*) begin
    m_axis_pkt_tvalid = 1'b0;

    case (state)

        ST_SEND_HEADER: begin
            m_axis_pkt_tvalid = 1'b1;
            ...
        end

    endcase
end
```

然后时序块中：

```verilog
if (m_axis_pkt_tvalid && m_axis_pkt_tready) begin
    header_index <= header_index + 1'b1;
end
```

假设现在：

```
state = SEND_HEADER
header_index = 2
```

组合逻辑于是输出：

```
TVALID = 1
TDATA  = Word2
```

如果：

```
TREADY = 0
```

没有 handshake，所以时序寄存器保持：

```
state        不变
header_index 不变
```

下一周期组合逻辑看到的仍然是：

```
SEND_HEADER + index=2
```

因此自然继续输出：

```
TVALID = 1
TDATA  = Word2
```

这正好满足 AXI4-Stream：

> `TVALID=1` 后，在真正 handshake 之前，不能因为 `TREADY=0` 就撤销当前数据。

为什么这种写法对你现在尤其合适

因为它让职责特别清楚：

```
组合逻辑：
“我现在应该向外展示什么？”

时序逻辑：
“这次传输真的发生后，我下一步去哪？”
```

比如：

```
SEND_HEADER
→ TVALID=1
→ TDATA=当前 Header Word

SEND_FIRST
→ TVALID=1
→ TDATA=first_sample_reg

SEND_PAYLOAD
→ TVALID=s_axis_iq_tvalid
→ TDATA=s_axis_iq_tdata
```

逻辑非常直观。

但是注意一个细节：不能根据 `TREADY` 去随意控制 `TVALID`

例如不要写成：

```verilog
if (m_axis_pkt_tready)
    m_axis_pkt_tvalid = 1'b1;
else
    m_axis_pkt_tvalid = 1'b0;
```

这是非常不好的 AXI 写法。

因为 Source 的 `TVALID` 不应该表示：

> “你愿意接，我才说我的数据有效。”

它应该表示：

> “我现在确实有一个有效 Word 等着传。”

所以在 `SEND_HEADER`：

```纯文本
有 Word 要发
→ TVALID 应该一直为 1
```

无论：

```纯文本
TREADY = 0
还是
TREADY = 1
```

都一样。

`TREADY` 只决定：

```纯文本
这一拍有没有真正完成传输
```

而不是决定数据有没有效。

`SEND_PAYLOAD` 稍微特殊一点

这里我们第一版采用直接流式透传：

```纯文本
IQ Source → Packetizer → Packet Output
```

所以可以让：

```verilog
m_axis_pkt_tvalid = s_axis_iq_tvalid;
```

同时：

```verilog
s_axis_iq_tready = m_axis_pkt_tready;
```

于是：

```纯文本
上游有有效 Sample
        ↓
输出就有有效 Payload

下游能接
        ↓
上游这个 Sample 才真正被消费
```

也就是：

```纯文本
s_axis_iq handshake
=
m_axis_pkt handshake
```

这一版很好理解。

那放到时序逻辑里行不行？

当然行。

比如你可以把：

```纯文本
m_axis_pkt_tvalid
m_axis_pkt_tdata
m_axis_pkt_tlast
```

都做成输出寄存器。

这种结构在工程上甚至经常很有价值，因为：

```
组合路径更短
输出更规整
更容易做高频
```

但代价是：

你必须自己管理“当前输出寄存器是否已经被消费，以及什么时候加载下一个 Word”。

也就是说，你需要类似：

```
output register occupied?
        ↓
如果下游接走
        ↓
才能加载下一个 Word
```

逻辑会明显复杂一些。

例如你不能简单地：

```verilog
always @(posedge aclk) begin
    m_axis_pkt_tvalid <= 1'b1;
    m_axis_pkt_tdata  <= next_word;
end
```

因为如果：

```
TREADY=0
```

你必须保证：

```
TDATA
TVALID
```

```
TLAST
```

继续保持原值。

这会引入“输出缓冲槽”这种概念。

因此对你现在这一版，我建议统一这样分

时序逻辑管理

```
state
data_seq
header_index
payload_count

first_sample_reg

snapshot_stream_id
snapshot_samples
snapshot_time_sec
snapshot_time_ps

pkt_busy
pkt_done
```

组合逻辑产生

```
s_axis_iq_tready

m_axis_pkt_tdata
m_axis_pkt_tvalid
m_axis_pkt_tlast
```

而：

```
m_axis_pkt_tkeep
```

因为永远固定：

```verilog
assign m_axis_pkt_tkeep = 4'b1111;
```

即可。

这不是唯一正确架构，但对你现在重新熟悉 RTL 来说，**最容易验证，也最不容易把 AXI 握手逻辑写乱。**

你后面写的时候可以一直用一句话检查：

> **组合逻辑负责“当前摆在接口上的是什么”；时序逻辑只在 handshake 后改变“下一拍该摆什么”。**

如果你把这个思想吃透，`m_axis_pkt_tvalid` 放组合逻辑就会非常自然。

你

状态机某状态的功能完成的done信号应该放到组合逻辑还是时序逻辑?

ChatGPT

如果你说的是这种：

> “某个状态的工作已经真正完成了，给其他模块/上层一个 `done` 脉冲”

那我更建议 `done` **做成时序逻辑产生的寄存器脉冲。**

但如果只是状态机内部用来判断“当前状态是不是完成条件成立了”，那么完全可以先定义一个**组合条件。**

这两个东西最好区分开。

以我们的 `difi_data_packetizer` 为例，接口规范已经规定 `pkt_done` 是“最后一个 Word 成功 handshake 后产生的 1-cycle pulse”。 DIFI_RTL_Interface_Spec_v1.0 所以这里 `pkt_done` 很适合放在时序逻辑中。

比如最后一个 Payload 正在发送时，内部可以先有一个组合意义上的完成条件：

```verilog
wire pkt_finish_cond;
```

```
assign pkt_finish_cond =
    m_axis_pkt_tvalid &&
    m_axis_pkt_tready &&
    m_axis_pkt_tlast;
```

这个信号表示：

“当前这一拍是否满足 Packet 完成条件？”

它可以是组合逻辑。

然后在时钟沿：

verilog

```
always @(posedge aclk) begin
    if (!aresetn) begin
        pkt_done <= 1'b0;
    end
    else begin
        pkt_done <= 1'b0;

        if (pkt_finish_cond)
            pkt_done <= 1'b1;
    end
end
```

于是 `pkt_done` 会变成非常干净的：

纯文本

```
0 0 0 0 1 0 0 0
        ↑
   Packet 完成
```

为什么外部 `done` 更推荐时序产生？主要有三个原因。

第一，它和“真正完成的时钟沿”严格对齐。

比如 AXI 的完成不是：

纯文本

```
TLAST = 1
```

就算完成，而是：

```
TVALID = 1
TREADY = 1
TLAST  = 1
```

在那个上升沿才算完成。

所以：

```
完成条件
   ↓
posedge
   ↓
pkt_done = 1
```

语义非常清楚。

第二，时序 `done` 不容易产生组合毛刺。

如果你直接：

```verilog
assign pkt_done =
    state == ST_SEND_PAYLOAD &&
    payload_count == snapshot_samples - 1 &&
    m_axis_pkt_tready;
```

它依赖很多组合信号。

理论上这些组合输入变化时，`pkt_done` 也会立即变化。

对于内部判断未必有问题，但作为模块对外事件信号，一般不如寄存器输出干净。

第三，它天然适合做“一拍脉冲”。

典型写法就是：

```verilog
pkt_done <= 1'b0;
```

```
if (完成事件)
    pkt_done <= 1'b1;
```

所以每次完成自动只高一拍。

但内部状态完成条件很适合组合逻辑

例如 `SEND_HEADER` 状态。

你可以定义：

```verilog
wire header_fire;
wire header_done;

assign header_fire =
    m_axis_pkt_tvalid &&
    m_axis_pkt_tready;

assign header_done =
    header_fire &&
    (header_index == 3'd6);
```

这里 `header_done` 的含义不是：

> “向系统输出一个完成脉冲。”

而只是：

> “当前这个周期，如果发生 handshake，那么 Word6 正好被送出去。”

然后时序状态机：

```verilog
always @(posedge aclk) begin
    case (state)

        ST_SEND_HEADER: begin
            if (header_done)
                state <= ST_SEND_FIRST;
            else if (header_fire)
                header_index <= header_index + 1'b1;
        end
```

```
    endcase
end
```

这种写法非常清楚。

所以你可以建立一个很实用的习惯：

```
xxx_done_cond / xxx_fire
        ↓
组合逻辑
        ↓
表示“当前完成条件是否成立”

真正对外的 xxx_done
        ↓
时序逻辑
        ↓
表示“刚刚发生了一次完成事件”
```

对我们的 Packetizer，建议你把几个信号分成这样：

```
组合条件：

iq_fire
= s_axis_iq_tvalid && s_axis_iq_tready

pkt_fire
= m_axis_pkt_tvalid && m_axis_pkt_tready

header_done_cond
= pkt_fire && header_index == 6

packet_done_cond
= pkt_fire && m_axis_pkt_tlast
```

而真正保存状态/输出事件的：

```
state
header_index
```

```
payload_count
data_seq
pkt_busy
pkt_done
```

放时序逻辑。

还有一个特别容易犯的错误：

不要仅仅因为“已经到了最后一个状态/最后一个数据”就产生 `done`。

例如：

```verilog
if (payload_count == snapshot_samples - 1)
    pkt_done <= 1'b1;
```

这是不够的。

因为此时可能：

```
TDATA  = 最后一个 Sample
TVALID = 1
TLAST  = 1
TREADY = 0
```

最后一个 Sample 还没有真正发出去。

所以正确条件必须包含 handshake：

```
最后一个 Sample
AND
TVALID
AND
TREADY
```

这才叫真正完成。

所以一句话总结：

“完成条件”可以组合地产生；“对外 done 事件”通常建议在完成条件对应的时钟沿寄存成一个 1-cycle pulse。

对我们现在的 `pkt_done`，我建议明确采用后一种。